Reporte

| | |
|---|---|
| **Nombre**: Emmanuel Carrillo Loera<br>Edgar Daniel del Ángel Hernández<br>Fatima Alejandra Cavazos Serrato<br>**Equipo:** 3 | **Matrícula**: 2067988<br>2094613<br>2099488 |
| **Nombre del curso:** Componentes de Lógica de Negocios | **Nombre del profesor**: Miguel Ángel Gómez Marroquín |
| **Modulo Netacad**: 3 | **Actividad**: Reporte de actividad fundamental 3 |
| **Fecha**: 13/10/2026 | |
| **Bibliografía (Formato APA)**<br>Archivo "Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx" proporcionado en la plataforma TEAMS<br>Archivo "03_Sazón_Urbano.docx" proporcionado en la plataforma TEAMS | |

**Objetivo**:

Convertir el análisis de negocio de Sazón Urbano de la Unidad 2 (BRD, historias de usuario, glosario y BPMN) en una propuesta de arquitectura de software coherente, que separe dominio, aplicación e infraestructura y que determine dónde vive cada regla de negocio antes de programar el sistema.

**Procedimiento**:

## Diseño del sistema

### Caso: Sazón Urbano

Esta actividad continúa el análisis de negocio que realizamos en la Unidad 2 para Sazón Urbano, un restaurante con servicio en mesa, para llevar y a domicilio. En aquella unidad definimos qué necesita el negocio: un documento de requerimientos (BRD), 20 historias de usuario con criterios de aceptación, un glosario de dominio y el proceso en BPMN. En esta unidad damos el paso siguiente: decidir dónde vive cada regla de negocio y cómo colaboran las partes del sistema para cumplirla, antes de escribir código.

Trabajamos en las nueve etapas que marca la guía de la actividad: preparar el caso, elegir el patrón de lógica de negocio, construir el mapa de dominio, diseñar el Service Layer, definir Repository y Unit of Work, separar Commands y Queries, revisar con SOLID, elaborar los diagramas y las decisiones de arquitectura (ADR) y, por último, validar la propuesta.

Usamos inteligencia artificial como apoyo para proponer, comparar y verificar, pero cada propuesta se contrastó con el BRD y con la guía de teams antes de aceptarla. Los errores que detectamos y las decisiones que tomamos como equipo están registrados en la bitácora de IA. Este informe resume el resultado de cada etapa y reúne las evidencias.

### Preparación del caso

En esta etapa convertimos el material de la Unidad 2 en la base de trabajo de la arquitectura. El BRD sigue siendo la guía: no cambiamos el alcance aprobado ni inventamos reglas para acomodar el diseño. Cuando algo de la Unidad 2 se contradecía o estaba incompleto, lo registramos como hallazgo y explicamos cómo lo tratamos.

**Proceso principal:** la gestión del pedido de Sazón Urbano, desde que el cliente ordena por cualquier canal hasta que el pedido se entrega, se cancela o la entrega no se completa.

Seleccionamos cinco casos de uso del BRD, que incluyen operaciones normales y al menos un camino de rechazo cada uno (salvo el cuarto):

- **CU-01 Registrar pedido:** rechaza un domicilio menor a 150 MXN (RN-01).
- **CU-02 Aplicar promoción:** rechaza una segunda promoción o una fuera de vigencia (RN-04).
- **CU-03 Cancelar pedido:** exige la autorización del Encargado si ya inició la preparación y genera merma (RN-02 y RN-03).
- **CU-04 Gestionar producto agotado:** cocina marca el producto y el mesero quita solo esa línea sin cancelar el pedido (RN-06).
- **CU-05 Asignar repartidor y registrar entrega:** exige motivo si la entrega falla (RN-11).

Para cada caso documentamos el actor, el objetivo, la información de entrada, las reglas relacionadas, el resultado esperado y su presencia en las historias de usuario o en el BPMN. Todos aparecen en las historias de usuario; el de promociones no aparece en el BPMN, lo cual se registró como hallazgo.

Registramos doce hallazgos. Los más importantes: Mesero y Caja son roles distintos (el mesero no cobra ni aplica descuentos y caja no sirve en mesa), aunque el BRD los agrupaba; el Cliente aparece en US-19 con permisos vacíos porque no usa el sistema; quitar una línea es una cancelación parcial; y el BPMN está incompleto y no muestra promociones ni alertas, lo cual corregiremos después de entregar esta actividad.

A continuación, se presenta la tabla de los 5 casos de uso:

Reporte

| ID | Caso de uso | Actor | Objetivo | Información de entrada | Reglas relacionadas | Resultado esperado |
|---|---|---|---|---|---|---|
| **CU-01** | Registrar pedido | Mesero (mesa) · Caja (teléfono, WhatsApp, domicilio) | Capturar el pedido completo y enviarlo a cocina | Canal, productos, cantidades, modificaciones, mesa o datos del cliente (teléfono y dirección si es domicilio), responsable | RN-01, RN-14 · NEC-11, NEC-12 | Pedido confirmado, con responsable y canal, enviado completo a las estaciones. Rechazo**:** domicilio < $150 o sin dirección/teléfono |
| **CU-02** | Aplicar promoción | Caja | Aplicar un descuento válido al pedido | Pedido, promoción seleccionada, fecha y hora | RN-04, RN-05, RN-09 | Pedido con una sola promoción vigente y total recalculado. Rechazo**:** fuera de vigencia o segunda promoción |
| **CU-03** | Cancelar pedido | Mesero o Caja (solicitan) · Encargado (autoriza) | Anular el pedido controlando la merma | Pedido, motivo, estado de las estaciones, autorización | RN-02, RN-03, RN-10 · NEC-04 | Pedido cancelado con motivo y responsable; merma si ya había preparación. Rechazo**:** requiere autorización del Encargado |
| **CU-04** | Gestionar producto agotado | **4a** Cocina (marca) · **4b** Mesero (quita línea) | Evitar pedidos incumplibles sin cancelarlos completos | 4a: producto o insumo · 4b: pedido y línea | RN-06, RN-12 · NEC-03 | 4a: producto no disponible y aviso inmediato · 4b: pedido sin esa línea, total recalculado, cancelación parcial registrada |
| **CU-05** | Asignar repartidor y registrar entrega | Encargado (asigna) · Repartidor (registra) | Entregar el pedido a domicilio con tiempos registrados | Pedido listo, repartidor disponible, hora de salida y entrega, motivo si falla | RN-11, RN-07 · NEC-06 | Pedido entregado con tiempos, o entrega no completada con motivo. Rechazo: entrega fallida |

## Tabla de hallazgos

| ID | Hallazgo | Tratamiento | Estado |
|---|---|---|---|
| H-01 | Mesero y Caja son roles distintos | Se ajusta el diseño y se actualiza la U2 | Supuesto a validar |
| H-02 | Cliente en US-19 sin usar el sistema | Rol con permisos vacíos | Resuelto |
| H-03 | Cancelación parcial vs quitar una línea | Quitar una línea **es** cancelación parcial | Resuelto |
| H-04 | BPMN con conexiones sin unir | Completar según el BRD | Pendiente |
| H-05 | BPMN sin promociones ni alertas | Agregar después de la actividad | Pendiente |
| H-06 | RN-08 y NEC-07 sin user story propia | Fuera de los 5 casos | Registrado |
| H-07 | Glosario dice que el Mesero cobra | Se corrige | Resuelto con H-01 |
| H-08 | RN-06 vs RN-12 al quitar una línea agotada | Prevalece RN-06 | Supuesto a validar |
| H-09 | El precio se conserva al capturar el pedido | Snapshot en Producto del pedido | Supuesto a validar |
| H-10 | Mecanismo con el que autoriza el Encargado | Se identifica como usuario autorizante | Supuesto a validar |
| H-11 | La U2 no definía permisos por acción | Se define la tabla 3 | Resuelto |
| H-12 | Estados del pedido salen del BPMN, no del BRD | Se documentan como derivados | Supuesto a validar |

## Selección del patrón de lógica de negocio

Domain Model como patrón principal, con Transaction Script solo para marcar un producto agotado y Table Module descartado en los cinco casos de uso (queda como candidato para los reportes).

Las razones que más pesaron fueron las siguientes. Primero, varias reglas dependen del estado del pedido por estación: cancelar sin autorización solo es posible si ninguna estación inició la preparación (RN-02), y si ya inició se exige al Encargado y se genera merma (RN-03). Con Transaction Script esa condición se repetiría en cada flujo; con Table Module no habría un lugar natural para el ciclo de vida del pedido. Segundo, las mismas reglas deben cumplirse igual en los tres canales de pedido. Tercero, las promociones cambian con frecuencia.

El costo que aceptamos es mayor estructura inicial que con scripts; lo acotamos con pocos aggregates y sin convertir cada tabla en una entidad. Transaction Script se justificó para marcar agotado porque es una operación lineal sin regla compleja.

Tabla comparativa de patrones

| **Criterio del caso** | **Transaction script** | **Table Module** | **Domain Model** | **Evidencia U2** |
|---|---|---|---|---|
| **Complejidad de reglas** | Con 6 reglas en 5 casos, cada script repetiría validaciones | Reglas atadas a datos, no a comportamiento | Cada regla vive en el objeto que la protege | RN-01, 02, 03, 04, 05, 12 · US-03, 07, 09 |
| **Cambios esperados** | Un cambio de promoción obliga a editar varios scripts | Cambio localizado por tabla, pero mezclado con SQL | El cambio queda en una sola clase | US-18 (vigencia configurable), entrevista (2x1 de martes aplicado “de memoria”), RN-07, RN-08, RN-15 |
| **Interacción entre conceptos** | Pedido, línea, estación, promoción, merma y entrega se cruzan dentro de un flujo largo | Cada módulo ve su tabla; la interacción queda fuera | Los objetos colaboran con sus reglas | RN-03 (cancelación + estación + merma + autorización), US-05 (recálculo del total) |
| **Estados e invariantes** | Hay que releer el estado en cada script | Sin modelo de ciclo de vida | Estado y transiciones protegidos | BPMN (¿Empezó preparación?, ¿Entrega exitosa?), RN-02, RN-04, RN-11 |

| | | | | |
|---|---|---|---|---|
| **Adicional 1: misma regla en varios canales** | Riesgo de duplicarla por canal | Posible, atada a la tabla | Una vez, sin importar el canal | BRD (mesa, teléfono/WhatsApp, domicilio) · US-01, US-02 |
| **Adicional 2: consultas y reportes** | Posible | Ideal | Pesado para solo leer | US-13, US-17 · NEC-08, NEC-09 |

Ventajas, riesgos y consecuencias

| **Patrón** | **Ventajas** | **Riesgos** | **Consecuencia en el caso** |
|---|---|---|---|
| **Transaction Script** | Rápido y claro para operaciones lineales (CU-04a) | Duplica lógica al crecer y mezcla reglas | Si se usara para el pedido, RN-02/03/12 se repetirían en cada flujo y cualquier cambio tocaría varios scripts |
| **Table Module** | Muy bueno para reportes y datos agregados | No modela el estado por estación ni las invariantes | Las reglas terminarían en servicios o en SQL, acopladas a las tablas |
| **Domain Model** | Reglas, estado y datos juntos; una regla, un lugar | Más clases y más estructura inicial; riesgo de sobreingeniería | Costo inicial mayor, que mitigamos limitando los aggregates (Pedido, Entrega) y exigiendo Repository y Unit of Work (Etapa 5) |

## Mapa de dominio

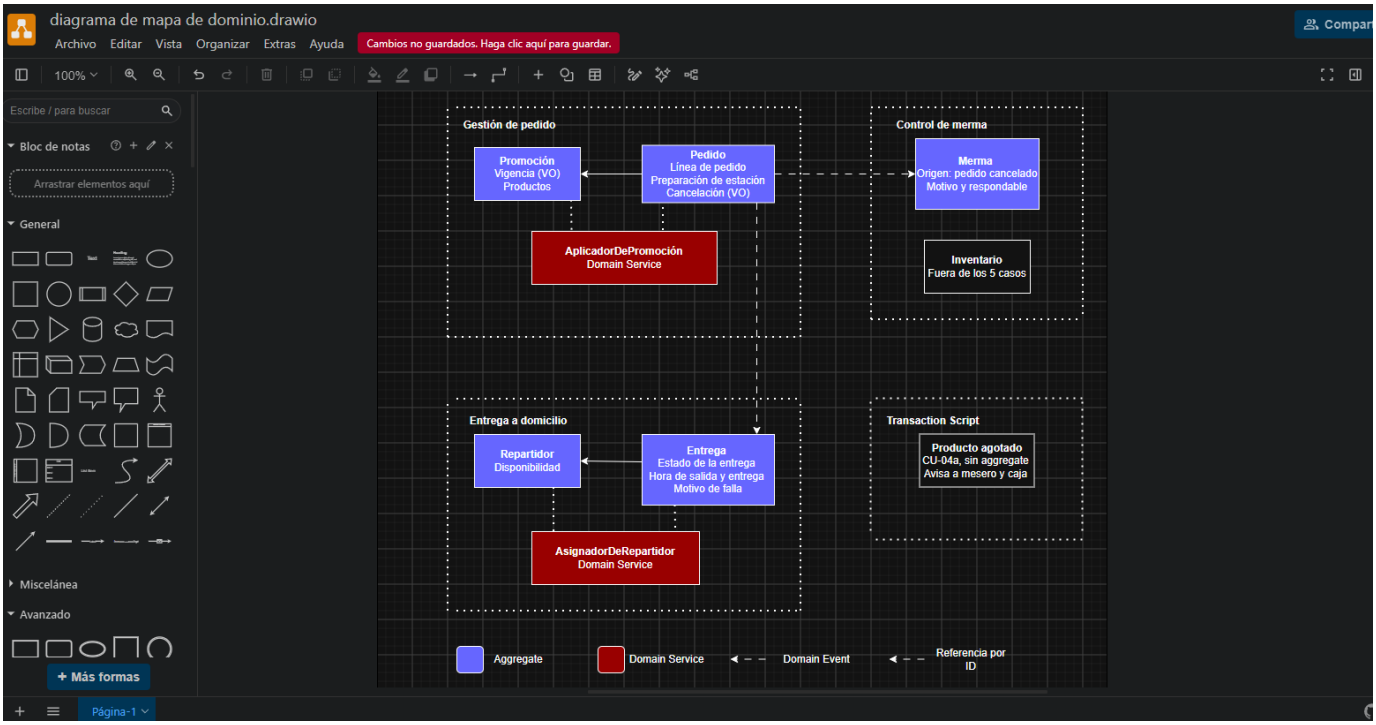
Aplicamos Domain-Driven Design: primero entender el negocio y después diseñar el software. En esta etapa identificamos los conceptos del negocio y los clasificamos como entidades, value objects, aggregates, domain services y domain events, usando el lenguaje del glosario de la Unidad 2 (lenguaje ubicuo). Un aggregate establece la frontera donde las reglas deben mantenerse consistentes, por lo que no convertimos cada tabla en una entidad.

Obtuvimos lo siguiente:

- **Cinco aggregates:** Pedido (con sus líneas, la preparación por estación y las cancelaciones), Promoción, Entrega, Repartidor y Merma. Tres tienen comportamiento real (Pedido, Entrega y Promoción) y dos son delgados a propósito (Repartidor y Merma). Los aggregates se conocen solo por su identificador.
- **Value objects:** Canal, Dinero, Cantidad, Modificación, Datos del cliente, Producto del pedido, Promoción aplicada, Cancelación, Vigencia, Estación de cocina y Responsable.
- **Dos domain services:** AplicadorDePromocion (coordina Pedido y Promoción) y AsignadorDeRepartidor (coordina Entrega y Repartidor).
- **Domain events:** PedidoConfirmado, PedidoCancelado, LineaDePedidoCancelada, PedidoListo, MermaRegistrada, ProductoMarcadoAgotado, y los de la entrega.

Documentamos una ficha por aggregate (raíz, responsabilidades, invariantes, eventos y casos de uso), actualizamos el glosario con términos nuevos y corregidos, y explicamos las decisiones descartadas: Cliente como entidad (no usa el sistema), Catálogo e Inventario como aggregates (fuera de los cinco casos), Ticket y Mesa (son lectura y un atributo) y Estación de cocina (es un value object).

Diagrama del mapa de dominio



Cada recuadro verde es un Aggregate y los límites punteados son áreas de responsabilidad. Los aggregates no se contienen entre sí: se conocen solo por ID (línea sólida). Cuando algo ocurre en uno y le interesa a otro, viaja como Domain Event (línea punteada): Pedido avisa a Merma cuando se cancela con preparación iniciada, y avisa a Entrega cuando queda listo.

Fichas de Aggregates

Pedido

| | |
|---|---|
| **Raíz** | `Pedido` |
| **Contiene** | Líneas de pedido, preparación por estación, promoción aplicada, cancelaciones, canal, responsable |
| **Responsabilidades** | Confirmarse, aplicar una promoción, cancelarse (total o parcial), quitar una línea, recalcular su total, marcarse listo |
| **Invariantes** | Domicilio < $150 no se confirma (RN-01) · Domicilio exige teléfono y dirección (US-02) · Registra responsable y canal (RN-14) · Una sola promoción, elegida explícitamente (RN-04, RN-05) · Sin autorización solo se cancela si **ninguna** estación inició (RN-02); si ya inició, exige al Encargado y genera merma (RN-03) · Modificar una línea requiere que su estación no haya iniciado, salvo autorización (RN-12) · El total siempre coincide con sus líneas y descuento |
| **Estados** | Capturado → Confirmado → En preparación → Listo → Entregado, Cancelado o Entrega no completada *(derivados del BPMN)* |
| **Eventos** | `PedidoConfirmado`, `PromocionAplicada`, `LineaDePedidoCancelada`, `PedidoCancelado`, `PedidoListo` |
| **Casos de uso** | CU-01, 02, 03, 04b, 05 |

Promoción

| | |
|---|---|
| **Raíz** | `Promoción` |
| **Contiene** | Vigencia, productos aplicables, condiciones |
| **Responsabilidades** | Decir si es aplicable en una fecha-hora y para ciertos productos, y calcular su descuento |
| **Invariantes** | Siempre tiene día, horario y productos definidos (US-18) · Solo el Gerente la configura (RN-09), un permiso que vive en la capa de aplicación |
| **Eventos** | Ninguno en los 5 casos |
| **Casos de uso** | CU-02 |

Entrega

| | |
|---|---|
| **Raíz** | `Entrega` (referencia `PedidoId` y `RepartidorId`) |
| **Contiene** | Estado, hora de salida, hora de entrega, motivo de falla |
| **Responsabilidades** | Asignarse, iniciar, completarse o fallar |
| **Invariantes** | Solo para pedidos a domicilio · No registra entrega sin hora de salida (RN-11) · Una entrega fallida exige motivo (RN-11) |
| **Eventos** | `RepartidorAsignado`, `EntregaIniciada`, `EntregaCompletada`, `EntregaFallida` |
| **Casos de uso** | CU-05 |

Repartidor

| | |
|---|---|
| **Raíz** | `Repartidor` |
| **Responsabilidades** | Conocer su disponibilidad |
| **Invariantes** | No se asigna a uno no disponible (US-14) |
| **Eventos** | Ninguno |

Merma

| | |
|---|---|
| **Raíz** | `Merma` (referencia `PedidoId`) |
| **Contiene** | Origen, motivo, responsable, fecha |
| **Invariantes** | Nace solo de una cancelación autorizada con preparación iniciada (RN-03) · Una vez registrada no se edita (trazabilidad, NEC-04) |
| **Eventos** | `MermaRegistrada` |
| **Casos de uso** | CU-03 |

## Service Layer

El Service Layer es la puerta de entrada de los casos de uso. Funciona como el director de una orquesta: decide qué pasos se realizan y en qué orden, pero no toca ningún instrumento, es decir, no contiene reglas de negocio. En esta etapa definimos los Application Services, qué recibe y devuelve cada uno, con quién colabora y qué reglas delega al dominio.

Diseñamos cinco métodos, uno por caso de uso, agrupados en tres servicios: PedidoApplicationService (registrar pedido, aplicar promoción y cancelar pedido), DisponibilidadApplicationService (gestionar producto agotado) y EntregaApplicationService (gestionar entrega). Mantuvimos los servicios separados para respetar el principio de responsabilidad única: la disponibilidad y las entregas no se mezclan con la gestión de pedidos.

Los dos métodos que cubren varios pasos reciben una solicitud que indica cuál se pide y la enrutan al comportamiento correcto; enrutar no es una decisión de negocio. Una primera versión tenía nueve métodos; la redujimos a cinco para ser fieles a la regla de un método por caso de uso y para no complicar el sistema.

Para cada método justificamos dónde vive cada regla. Por ejemplo, el monto mínimo de domicilio, la autorización de cancelaciones y la no acumulación de promociones viven en el Pedido; la vigencia y los productos de una promoción viven en la Promoción; y el control de qué rol puede hacer cada acción vive en la capa de aplicación porque es seguridad por rol y no una regla de dominio.

Tabla de application services

| **Servicio** | **Método (CU)** | **Entrada** | **Colaboradores** | **Salida** | **Reglas que delega** |
|---|---|---|---|---|---|
| `PedidoApplicationService` | `registrarPedido` (CU-01) | Canal, líneas (producto, cantidad, modificaciones), datos del cliente, mesa, responsable | Control de acceso, `PedidoRepository`, Unit of Work, publicador de eventos | Pedido confirmado (id, total, estado). Errores: `MontoMinimoDomicilioNoAlcanzado`, `DatosDeDomicilioIncompletos` | Pedido: RN-01, RN-14, datos obligatorios por canal (US-02), total coherente |
| `PedidoApplicationService` | `aplicarPromocion` (CU-02) | Id del pedido, id de la promoción elegida, fecha-hora, responsable | Control de acceso, `PedidoRepository`, `PromocionRepository`, `AplicadorDePromocion`, Unit of Work | Pedido con descuento y total recalculado. Errores: `PromocionFueraDeVigencia`, `PromocionNoAplicable`, `PedidoYaTienePromocion` | Promoción (vía `AplicadorDePromocion`): vigencia y productos · Pedido: RN-04 |
| `PedidoApplicationService` | `cancelarPedido` (CU-03) | Id del pedido, motivo, solicitante, autorizador (Encargado, solo si ya inició preparación) | Control de acceso, `PedidoRepository`, `MermaRepository`, Unit of Work, publicador | Cancelación confirmada, indicando si generó merma. Error: `CancelacionRequiereAutorizacion` | Pedido: RN-02, RN-03 · Merma: se crea al recibir `PedidoCancelado` |
| `DisponibilidadApplicationService` | `gestionarProductoAgotado` (CU-04) | Una de dos solicitudes (ver abajo) + responsable | Control de acceso, `DisponibilidadRepository`, `ConsultasDeDisponibilidad` *(solo consulta)*, `PedidoRepository`, notificador, Unit of Work | producto no disponible con aviso, o pedido actualizado. **Errores:** `ProductoNoEstaAgotado`, `LineaNoExiste` | Marcar agotado**:** ninguna, es el **Transaction Script** · **Quitar línea:** Pedido (RN-06, RN-12, recálculo) |
| `EntregaApplicationService` | `gestionarEntrega` (CU-05) | Una de cuatro solicitudes (ver abajo) + responsable | Control de acceso, `PedidoRepository`, `RepartidorRepository`, `EntregaRepository`, `AsignadorDeRepartidor`, Unit of Work, publicador | entrega asignada, en ruta, completada o fallida. Errores: `RepartidorNoDisponible`, `PedidoNoEsADomicilio`, `PedidoNoEstaListo`, `EntregaSinSalidaRegistrada`, `MotivoDeFallaObligatorio` | Entrega / AsignadorDeRepartidor: disponibilidad, solo domicilio · Entrega: RN-11 · Pedido: estado listo y cierre |

Tabla de solicitudes que enruta cada método

| **Método** | **Solicitud** | **Quién la pide** | **Datos** | **Efecto** |
|---|---|---|---|---|
| `gestionarProductoAgotado` | Marcar producto agotado | Cocina | Producto o insumo | El producto deja de estar disponible y se avisa a Mesero y Caja |
| | Quitar línea agotada | Mesero | Id del pedido, id de la línea | El pedido sigue sin esa línea y con total recalculado |
| `gestionarEntrega` | Asignar repartidor | Encargado | Id del pedido, id del repartidor | Se crea la entrega vinculada al repartidor |
| | Registrar salida | Repartidor | Id de la entrega | Entrega en ruta, con hora de salida |
| | Confirmar entrega | Repartidor | Id de la entrega | Entrega completada y pedido entregado |
| | Registrar entrega fallida | Repartidor | Id de la entrega, motivo | Entrega fallida y pedido en “entrega no completada” |

## Repository y Unit of Work

El patrón Repository separa la lógica de negocio de la forma en que se guardan y recuperan los datos. El dominio pide “dame el pedido 1001” sin saber si detrás hay SQL, una API o archivos. La Unit of Work decide qué cambios se guardan juntos: o se confirma todo, o no se guarda nada. En esta etapa definimos ambas piezas.

Lo que hicimos fue:

- Seis repositorios con nueve métodos en total, todos usados por los cinco casos. Las interfaces viven en el dominio y hablan en términos del negocio; las implementaciones viven en infraestructura (en memoria para el código base, y una relacional como posibilidad).
- Un contrato común: obtener por id siempre lanza un error de negocio si el objeto no existe, sin importar la implementación.
- Una unidad de trabajo por solicitud: inicia al comenzar el método del servicio, se confirma al final y, si algo falla, se deshace todo y no se publica ningún evento. Los eventos se publican solo después de confirmar.

- Identificamos qué debe guardarse junto: al cancelar con preparación iniciada, el Pedido y la Merma; al cerrar una entrega, la Entrega, el Pedido y el Repartidor.
- Distinguimos errores de negocio (por ejemplo, monto mínimo no alcanzado) de errores técnicos (por ejemplo, una caída de conexión, algo que el BRD advierte por la red intermitente).

Decidimos no implementar protección contra concurrencia para mantener el diseño simple; es un riesgo aceptado, documentado en los ADR.

Tabla de Repositorios

| **Repository** | **Necesidad del dominio** | **Métodos propuestos** | **Objeto que recupera o guarda** |
|---|---|---|---|
| `PedidoRepository` | Recuperar un pedido completo para cancelarlo, aplicarle promoción, quitarle una línea o entregarlo. Guardar el pedido nuevo o modificado | `obtenerPorId(pedidoId)` · `guardar(pedido)` | `Pedido` completo (con líneas, preparación por estación y cancelaciones) |
| `PromocionRepository` | Conocer una promoción para saber si aplica | `obtenerPorId(promocionId)` | `Promoción` |
| `EntregaRepository` | Recuperar una entrega en curso y guardar su avance | `obtenerPorId(entregaId)` · `guardar(entrega)` | `Entrega` |
| `RepartidorRepository` | Saber si un repartidor está disponible y actualizar su disponibilidad | `obtenerPorId(repartidorId)` · `guardar(repartidor)` | `Repartidor` |
| `MermaRepository` | Registrar la pérdida generada por una cancelación | `guardar(merma)` | `Merma` |
| `DisponibilidadRepository` | Marcar un producto como agotado | `marcarAgotado(producto, responsable, fechaHora)` | Registro de disponibilidad de producto *(no es aggregate: lo maneja el Transaction Script)* |

Tabla de Unit of Work

| Método de servicio | Cambios que deben completarse juntos | Si algo falla |
|---|---|---|
| `registrarPedido` | `Pedido` nuevo (confirmado, con sus líneas) | No se guarda nada y **no se envía a cocina** |
| `aplicarPromocion` | `Pedido` con promoción y total recalculado | El pedido queda como estaba |
| `cancelarPedido` | `Pedido` cancelado + `Merma` (si ya había preparación) | Ni cancelación ni merma: nunca queda un pedido cancelado sin su merma (RN-03) |
| `gestionarProductoAgotado` | *Marcar:* disponibilidad del producto · *Quitar línea:* `Pedido` con total recalculado | Se deshace y **no se avisa** a Mesero y Caja |
| `gestionarEntrega` | *Asignar:* `Entrega` nueva + `Repartidor` no disponible · *Salida:* `Entrega` · *Confirmar o fallar:* `Entrega` + `Pedido` + `Repartidor` disponible | Se deshace todo y la entrega queda en su estado anterior |

## Commands y Queries: CQRS básico

CQRS (Command Query Responsibility Segregation) propone separar las operaciones que cambian el sistema (commands) de las que solo consultan información (queries). Un command ordena que algo cambie y aplica reglas; un query pregunta cómo está algo y nunca modifica nada. En esta etapa aplicamos una versión básica de esa separación.

Lo que hicimos fue lo siguiente:

- Nueve commands, uno por cada operación que cambia el estado, cada uno con su handler: RegistrarPedido, AplicarPromocion, CancelarPedido, MarcarProductoAgotado, QuitarLineaAgotada, AsignarRepartidor, RegistrarSalidaDeEntrega, ConfirmarEntrega y RegistrarEntregaFallida.

- Cuatro queries: resumen del pedido, menú disponible, promociones vigentes y repartidores disponibles. Dependen de interfaces de lectura y devuelven DTO (objetos simples que solo transportan datos), nunca el aggregate.
- Una sola base de datos: la separación es en el código, porque separar almacenes de lectura y escritura sería complejidad sin beneficio para un restaurante único.
- Marcar un producto agotado queda como un command simple, sin read model propio.

Tabla de Commands

| **Tipo** | **Nombre** | **Datos de entrada** | **Handler y dependencias** | **Resultado** | **Justificación** |
|---|---|---|---|---|---|
| Command | `RegistrarPedido` | Canal, líneas, datos del cliente, mesa, responsable | `RegistrarPedidoHandler` → `PedidoRepository`, Unit of Work, publicador | Id, total y estado del pedido | Crea el pedido y aplica RN-01 y RN-14 (CU-01) |
| Command | `AplicarPromocion` | Id del pedido, id de la promoción, fecha-hora, responsable | `AplicarPromocionHandler` → `PedidoRepository`, `PromocionRepository`, `AplicadorDePromocion`, Unit of Work | Pedido con descuento y total recalculado | Cambia el total y exige una sola promoción (RN-04) |
| Command | `CancelarPedido` | Id del pedido, motivo, solicitante, autorizador (si aplica) | `CancelarPedidoHandler` → `PedidoRepository`, `MermaRepository`, Unit of Work, publicador | Cancelación confirmada, indicando si hubo merma | Aplica RN-02 y RN-03 |
| Command | `MarcarProductoAgotado` | Producto o insumo, responsable | `MarcarProductoAgotadoHandler` → `DisponibilidadRepository`, notificador | Producto no disponible | Cambia la disponibilidad y avisa a Mesero y Caja |
| Command | `QuitarLineaAgotada` | Id del pedido, id de la línea, responsable | `QuitarLineaAgotadaHandler` → `PedidoRepository`, **`ConsultasDeDisponibilidad`**, Unit of Work | Pedido sin la línea y con total recalculado | Cancelación parcial (RN-06, RN-12) |
| Command | `AsignarRepartidor` | Id del pedido, id del repartidor, responsable | `AsignarRepartidorHandler` → `PedidoRepository`, `RepartidorRepository`, `EntregaRepository`, | Entrega con repartidor asignado | Crea la entrega y marca al repartidor no disponible |

| | | | | | |
|---|---|---|---|---|---|
| | | | `AsignadorDeRepartidor`, Unit of Work | | |
| Command | `RegistrarSalidaDeEntrega` | Id de la entrega, responsable | `RegistrarSalidaHandler` → `EntregaRepository`, Unit of Work | Entrega en ruta, con hora de salida | RN-11 |
| Command | `ConfirmarEntrega` | Id de la entrega, responsable | `ConfirmarEntregaHandler` → `EntregaRepository`, `PedidoRepository`, `RepartidorRepository`, Unit of Work, publicador | Entrega completada y pedido entregado | RN-11 y cierre del pedido |
| Command | `RegistrarEntregaFallida` | Id de la entrega, motivo, responsable | `RegistrarEntregaFallidaHandler` → mismas dependencias que `ConfirmarEntrega` | Pedido en "entrega no completada" | RN-11: el motivo es obligatorio |

Tabla de Queries

| **Tipo** | **Nombre** | **Datos de entrada** | **Handler y dependencias** | **Resultado** | **Justificación** |
|---|---|---|---|---|---|
| Query | `ConsultarResumenDelPedido` | Id del pedido | `ConsultarResumenHandler` → `ConsultasDePedidos` | `ResumenDePedidoDto` | Mesero y Caja ven el pedido antes de aplicar promoción o quitar una línea. Es el ejemplo del carrito que dio el profe |
| Query | `ConsultarMenuDisponible` | Fecha-hora | `ConsultarMenuHandler` → `ConsultasDeDisponibilidad` | Lista de `ProductoDisponibleDto` | Resuelve US-04: lo agotado "deja de mostrarse" sin meter una regla extra en `RegistrarPedido` |
| Query | `ConsultarPromocionesVigentes` | Fecha-hora | `ConsultarPromocionesHandler` → `ConsultasDePromociones` | Lista de `PromocionVigenteDto` | Caja elige la promoción de forma explícita (RN-05) |
| Query | `ConsultarRepartidoresDisponibles` | Ninguno | `ConsultarRepartidoresHandler` → `ConsultasDeRepartidores` | Lista de `RepartidorDisponibleDto` | El Encargado asigna uno disponible (US-14) |

Tabla de DTO (modelo de lectura)

| DTO | Campos | Por qué no es el modelo de escritura |
|---|---|---|
| `ResumenDePedidoDto` | Id, canal, mesa o cliente (texto), líneas (producto, cantidad, modificaciones, subtotal), promoción (nombre y descuento), total, estado (texto), responsable (nombre) | Es plano y está listo para mostrar: no tiene métodos ni reglas, ni expone la preparación por estación ni las cancelaciones internas. El aggregate `Pedido` protege reglas; este DTO solo muestra datos |
| `ProductoDisponibleDto` | Id, nombre, precio, disponible (sí/no) | El catálogo no es un aggregate del dominio (decisión 3 de la Etapa 3) |
| `PromocionVigenteDto` | Id, nombre, productos, horario (texto) | Muestra la promoción sin su lógica de vigencia |
| `RepartidorDisponibleDto` | Id, nombre | Solo lo necesario para elegir |

DTO= Data Transfer Object. En nuestro diseño, es lo que devuelven las Queries (por ejemplo, ResumenDePedidoDto). Por eso no usamos el aggregate Pedido para mostrar datos: el aggregate protege reglas y el DTO solo muestra información.

## Revisión con SOLID

Antes de cerrar los diagramas revisamos la propuesta completa con los cinco principios SOLID, con el objetivo de detectar clases que hicieran demasiado, dependencias innecesarias o reglas que obligaran a modificar código existente.

Obtuvimos siete hallazgos: cuatro confirman buenas decisiones y tres señalaron correcciones, que aplicamos.

- **S (responsabilidad única):** confirmamos que los tres servicios están separados; el servicio solo enruta y cada paso vive en su handler.
- **O (abierto/cerrado), corrección:** la Promoción calcula su propio descuento, de modo que un tipo nuevo de promoción (por ejemplo 2x1) se agrega como variante sin editar Pedido ni servicios.
- **L (sustitución de Liskov), corrección:** definimos un contrato común para los repositorios, de modo que cambiar de implementación no altere el comportamiento.

- **I (segregación de interfaces), corrección:** separamos la lectura de la escritura de disponibilidad: el repositorio solo escribe y la consulta “está agotado” vive en una interfaz de lectura.
- **D (inversión de dependencias):** confirmamos que dominio y aplicación dependen solo de abstracciones; el SQL queda en las implementaciones.

Dejamos documentado un riesgo: el Pedido concentra mucho comportamiento. No lo dividimos porque partirlo rompería la consistencia que protege como aggregate; lo mitigamos con value objects y con el servicio de dominio AplicadorDePromocion.

Tabla de revisión SOLID

| Principio | Pregunta propuesta | Respuesta con evidencia de nuestro diseño |
|---|---|---|
| **S** | ¿Cada clase o servicio tiene una razón principal para cambiar? | Sí: tres servicios separados (`Pedido`, `Disponibilidad`, `Entrega`) y un Handler por Command. Vigilar `Pedido`, que concentra varias responsabilidades |
| **O** | ¿Se puede agregar una variante de regla sin editar una cadena de condiciones? | Riesgo con **promociones**: hoy hay un solo tipo, pero la entrevista menciona un 2x1 y US-18 habla de “condiciones” |
| **L** | ¿Las implementaciones se pueden sustituir sin cambiar el comportamiento? | Los repositorios en memoria y relacionales deben comportarse igual, y el contrato no estaba escrito |
| **I** | ¿Cada consumidor depende solo de lo que usa? | `DisponibilidadRepository` mezclaba lectura y escritura, pero `QuitarLineaAgotada` solo necesita leer |
| **D** | ¿Aplicación y dominio dependen de abstracciones? | Sí: solo interfaces (repositorios, Unit of Work, publicador, notificador, control de acceso). No hay SQL en el modelo |

Hallazgos de la revisión con SOLID

H1 (S, responsabilidad única): confirma una buena decisión

- Evidencia: los Application Services de disponibilidad y de entregas están separados del servicio de pedidos.
- Impacto: ningún servicio hace “de todo”; marcar un producto agotado no se mezcla con la gestión de pedidos.
- Cambio realizado: ninguno.

H2 (S, responsabilidad única): riesgo mitigado

- Evidencia: gestionarEntrega y gestionarProductoAgotado reciben varias solicitudes y podrían crecer.
- Impacto: si tuvieran la lógica adentro serían servicios que hacen de todo.
- Cambio realizado: el servicio solo enruta, y cada paso vive en su Handler (Etapa 6).

H3 (O, abierto/cerrado): corrección

- Evidencia: no estaba definido quién calcula el descuento de una promoción. Si lo hiciera un servicio o el Pedido, cada tipo nuevo (2x1, porcentaje) sumaría condiciones.
- Impacto: agregar un tipo de promoción obligaría a editar código que ya funciona.
- Cambio realizado: la Promoción calcula su propio descuento. Un tipo nuevo se agrega como variante de Promoción, sin tocar Pedido ni el servicio.

H4 (L, sustitución de Liskov): corrección

- Evidencia: el repositorio en memoria y el relacional podrían comportarse distinto (uno devuelve vacío y otro lanza error si el pedido no existe).
- Impacto: cambiar de implementación alteraría el comportamiento del sistema.
- Cambio realizado: contrato explícito. obtenerPorId siempre lanza un error de negocio si el objeto no existe, en cualquier implementación.

H5 (I, segregación de interfaces): corrección

- Evidencia: DisponibilidadRepository tenía estaAgotado y marcarAgotado, pero QuitarLineaAgotada solo necesita consultar.
- Impacto: un consumidor dependía de un método de escritura que no usa.
- Cambio realizado: DisponibilidadRepository conserva solo marcarAgotado, y estaAgotado pasa a la interfaz de lectura ConsultasDeDisponibilidad.

H6 (I, segregación de interfaces): confirma una buena decisión

- Evidencia: los handlers de consulta dependen de interfaces de lectura (ConsultasDe…) y no de los repositorios.
- Impacto: las consultas no reciben métodos de escritura como guardar.
- Cambio realizado: ninguno.

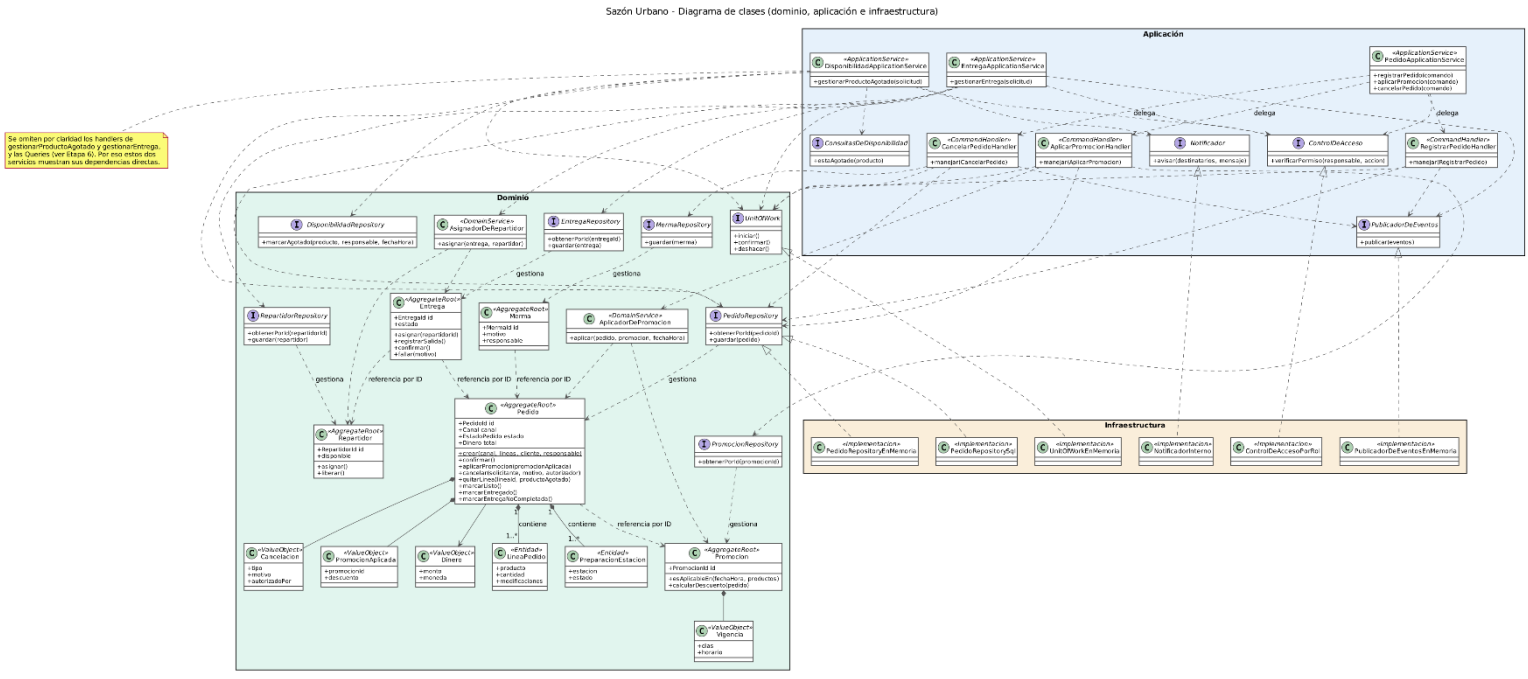
H7 (D, inversión de dependencias): confirma una buena decisión

- Evidencia: el dominio y los handlers solo conocen interfaces; las implementaciones (SQL o memoria) están en infraestructura.
- Impacto: migrar de SQL Server a PostgreSQL solo toca infraestructura.
- Cambio realizado: ninguno.

## Diagramas UML

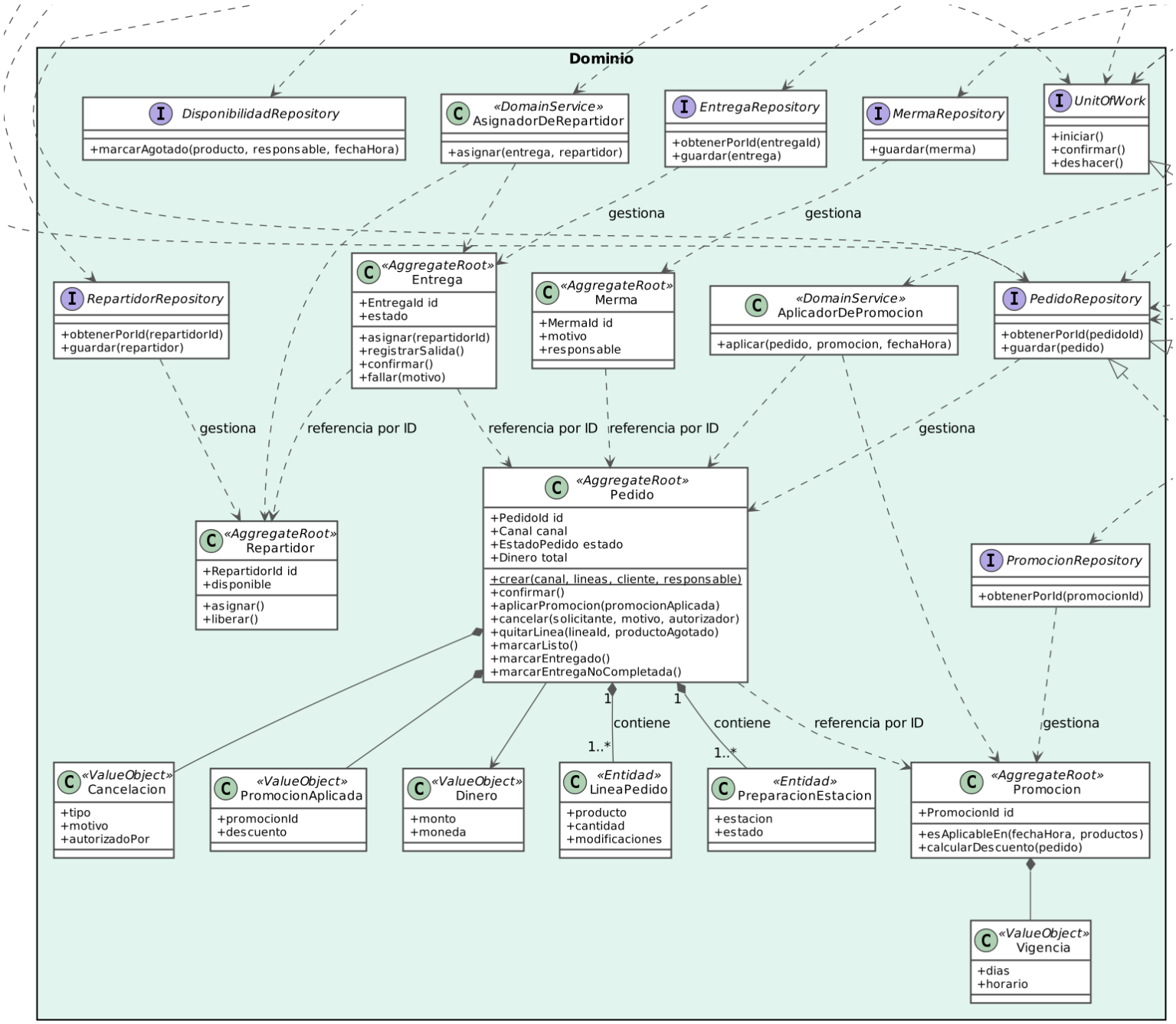
Elaboramos el diagrama de clases completo y dos diagramas de secuencia con PlantUML. Los archivos fuente están en el repositorio junto con sus exportaciones en PNG. Verificamos que ambos tipos de diagrama representen la misma arquitectura: los quince participantes de las secuencias existen en el diagrama de clases y no hay clases sin relaciones.

El diagrama distingue visualmente las tres capas con color: dominio (verde), aplicación (azul) e infraestructura (naranja). Incluye los aggregates con sus entidades y value objects, los domain services, las interfaces de Repository y la Unit of Work, los Application Services y handlers, los puertos (control de acceso, publicador, notificador y consulta de disponibilidad) y las implementaciones de infraestructura. Las flechas de dependencia apuntan hacia las interfaces, no hacia las implementaciones. Por claridad se dibujaron tres de los nueve handlers y no se dibujan las queries. La imagen se presenta completa y ampliada por capas para que sea legible.

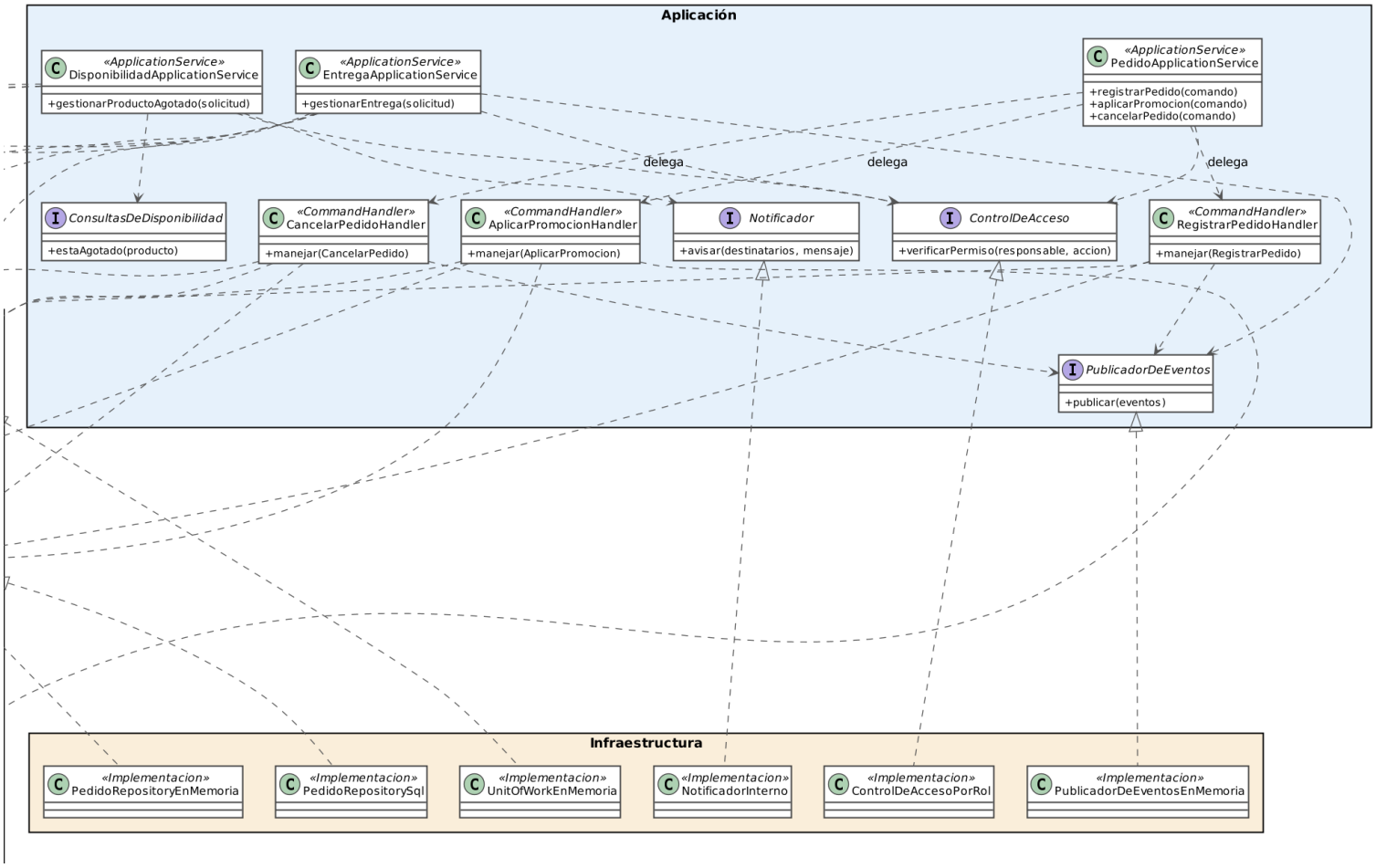



La vista general muestra cómo se relacionan las tres capas. Para poder leerlo con comodidad, las dos figuras siguientes amplían la capa de dominio y las capas de aplicación e infraestructura.

Reporte

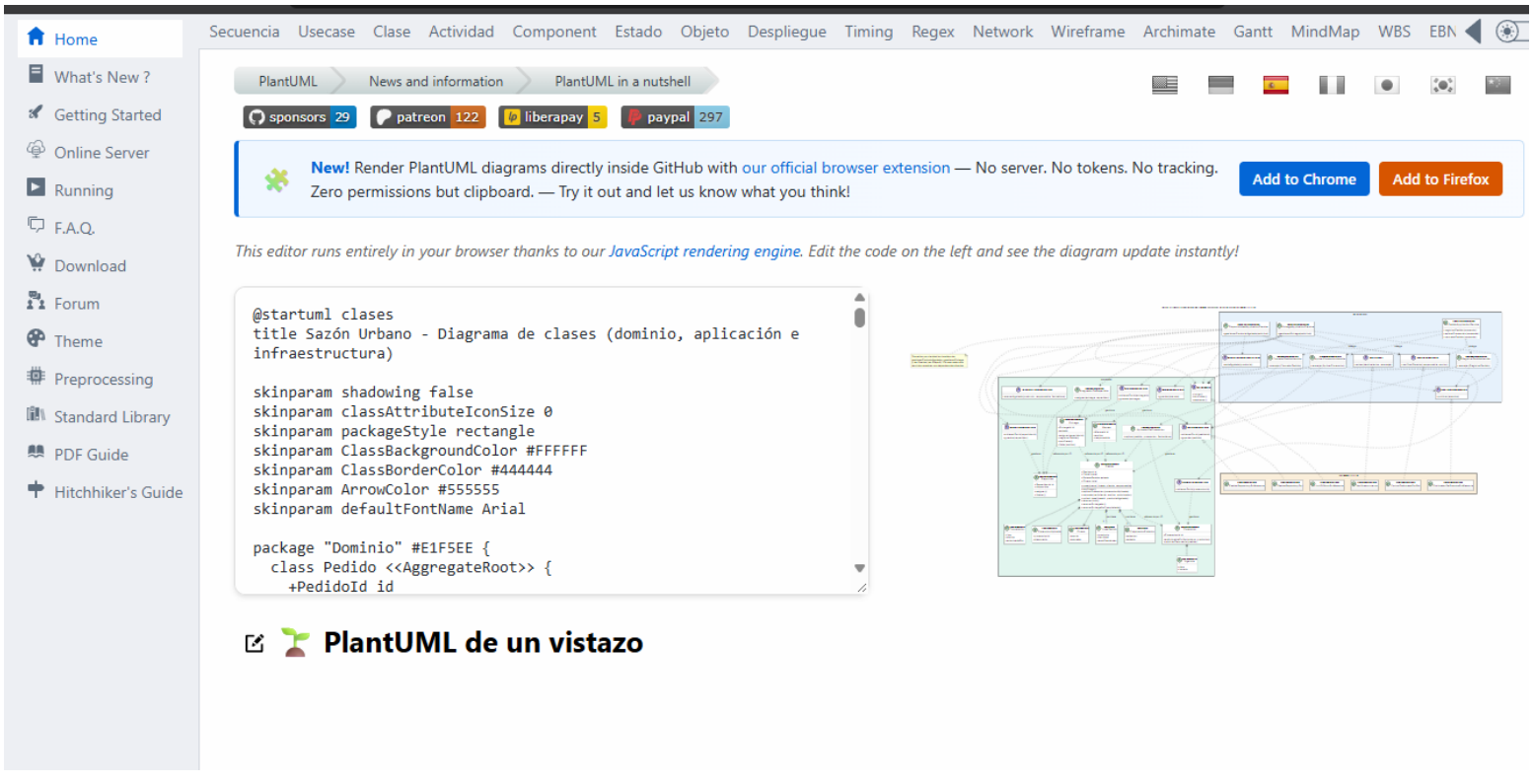

Dominio
DisponibilidadRepository
+marcarAgotado(producto, responsable, fechaHora)
«DomainService»
AsignadorDeRepartidor
+asignar(entrega, repartidor)
EntregaRepository
+obtenerPorId(entregaId)
+guardar(entrega)
MermaRepository
+guardar(merma)
UnitOfWork
+iniciar()
+confirmar()
+deshacer()
gestiona
gestiona
RepartidorRepository
+obtenerPorId(repartidorId)
+guardar(repartidor)
«AggregateRoot»
Entrega
+EntregaId id
+estado
+asignar(repartidorId)
+registrarSalida()
+confirmar()
+fallar(motivo)
«AggregateRoot»
Merma
+MermaId id
+motivo
+responsable
«DomainService»
AplicadorDePromocion
+aplicar(pedido, promocion, fechaHora)
PedidoRepository
+obtenerPorId(pedidoId)
+guardar(pedido)
gestiona
referencia por ID
referencia por ID
referencia por ID
gestiona
«AggregateRoot»
Repartidor
+RepartidorId id
+disponible
+asignar()
+liberar()
«AggregateRoot»
Pedido
+PedidoId id
+Canal canal
+EstadoPedido estado
+Dinero total
+crear(canal, lineas, cliente, responsable)
+confirmar()
+aplicarPromocion(promocionAplicada)
+cancelar(solicitante, motivo, autorizador)
+quitarLinea(lineaId, productoAgotado)
+marcarListo()
+marcarEntregado()
+marcarEntregaNoCompletada()
PromocionRepository
+obtenerPorId(promocionId)
1
1
contiene
contiene
1..*
1..*
referencia por ID
gestiona
«ValueObject»
Cancelacion
+tipo
+motivo
+autorizadoPor
«ValueObject»
PromocionAplicada
+promocionId
+descuento
«ValueObject»
Dinero
+monto
+moneda
«Entidad»
LineaPedido
+producto
+cantidad
+modificaciones
«Entidad»
PreparacionEstacion
+estacion
+estado
«AggregateRoot»
Promocion
+PromocionId id
+esAplicableEn(fechaHora, productos)
+calcularDescuento(pedido)
«ValueObject»
Vigencia
+dias
+horario

Reporte


Aplicación
«ApplicationService»
DisponibilidadApplicationService
+gestionarProductoAgotado(solicitud)
«ApplicationService»
EntregaApplicationService
+gestionarEntrega(solicitud)
«ApplicationService»
PedidoApplicationService
+registrarPedido(comando)
+aplicarPromocion(comando)
+cancelarPedido(comando)
delega
delega
delega
ConsultasDeDisponibilidad
+estaAgotado(producto)
«CommandHandler»
CancelarPedidoHandler
+manejar(CancelarPedido)
«CommandHandler»
AplicarPromocionHandler
+manejar(AplicarPromocion)
Notificador
+avisar(destinatarios, mensaje)
ControlDeAcceso
+verificarPermiso(responsable, accion)
«CommandHandler»
RegistrarPedidoHandler
+manejar(RegistrarPedido)
PublicadorDeEventos
+publicar(eventos)
Infraestructura
«Implementacion»
PedidoRepositoryEnMemoria
«Implementacion»
PedidoRepositorySql
«Implementacion»
UnitOfWorkEnMemoria
«Implementacion»
NotificadorInterno
«Implementacion»
ControlDeAccesoPorRol
«Implementacion»
PublicadorDeEventosEnMemoria

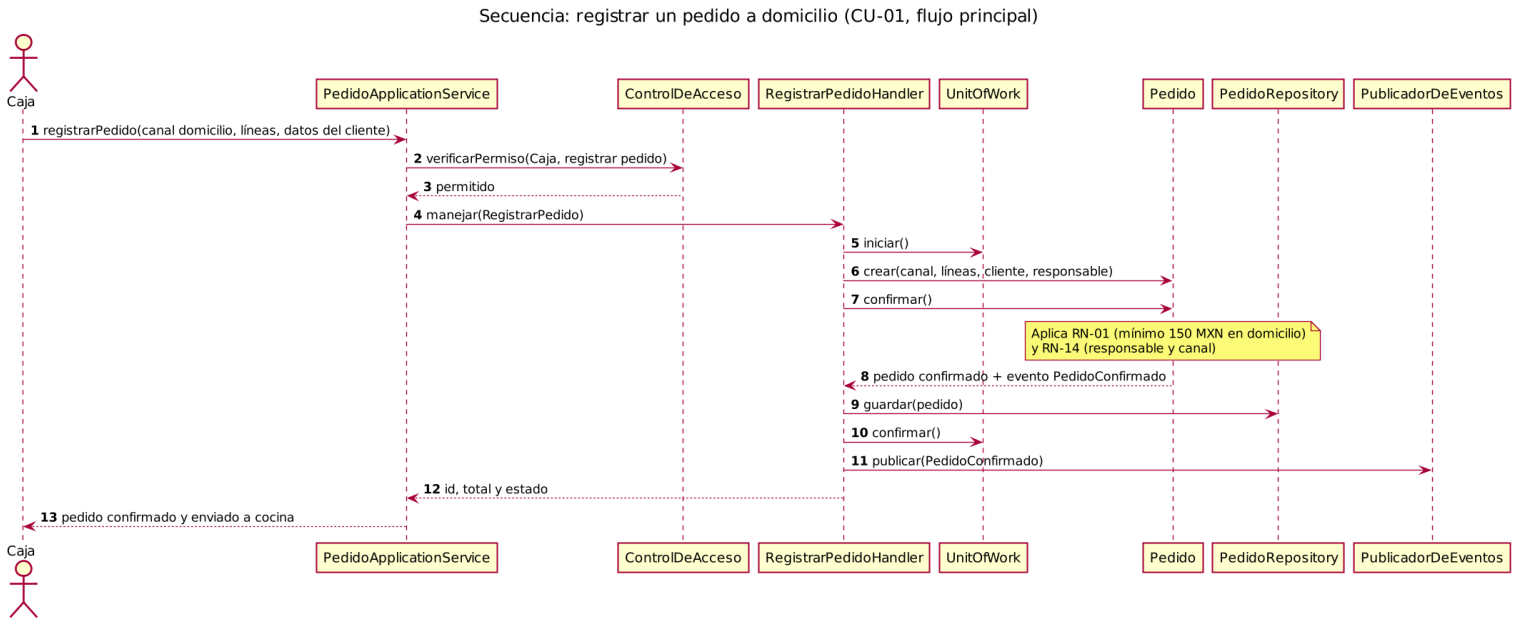
Reporte

Código del diagrama de clases en plantuml



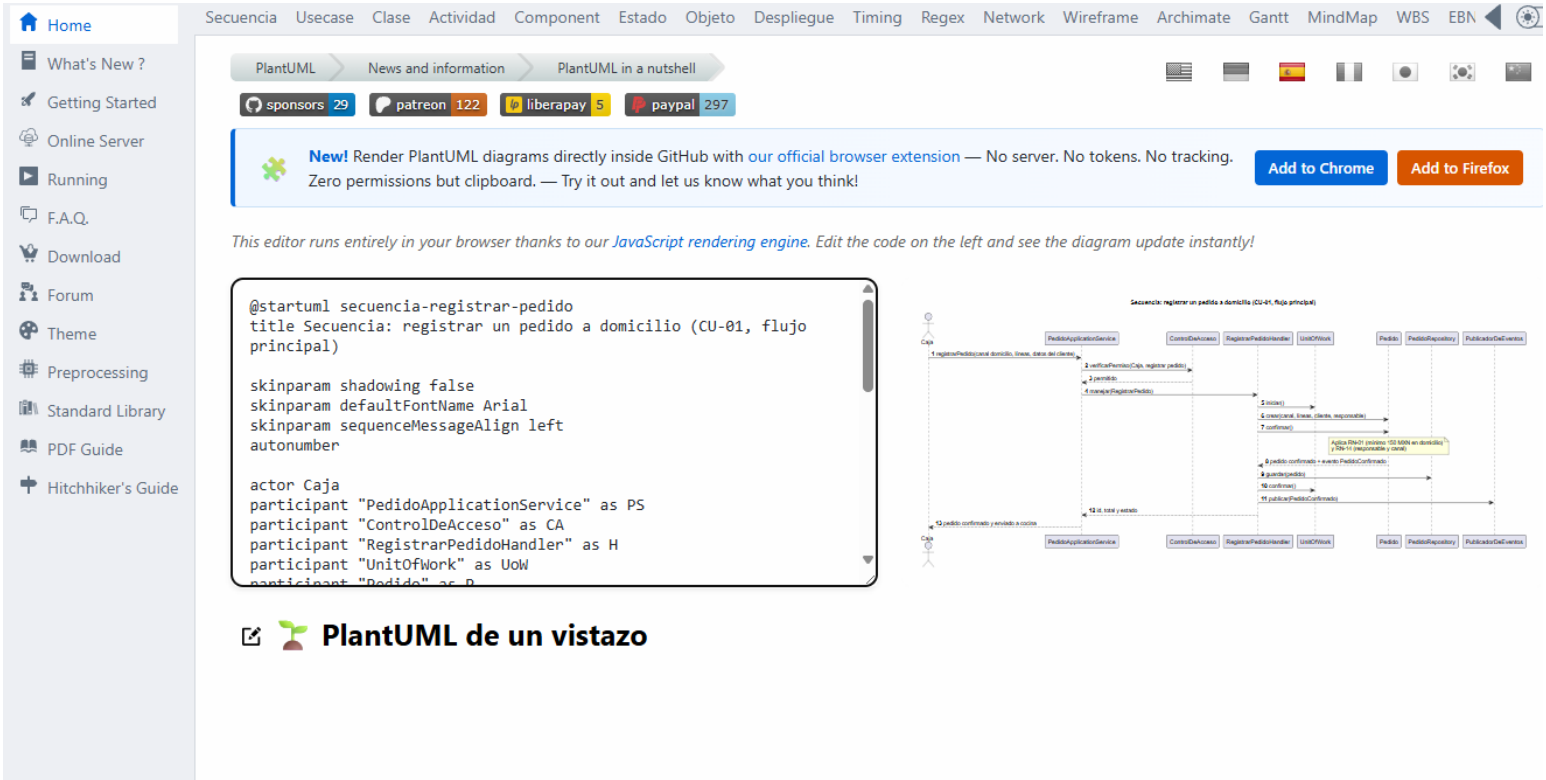
## Diagrama de secuencia

**Flujo principal (CU-01):** registrar un pedido a domicilio. Muestra el patrón de orquestación completo: la Caja solicita el registro, el servicio verifica el permiso, el handler abre la unidad de trabajo, el Pedido aplica RN-01 y RN-14, se guarda, se confirma y, solo entonces, se publica el evento PedidoConfirmado.

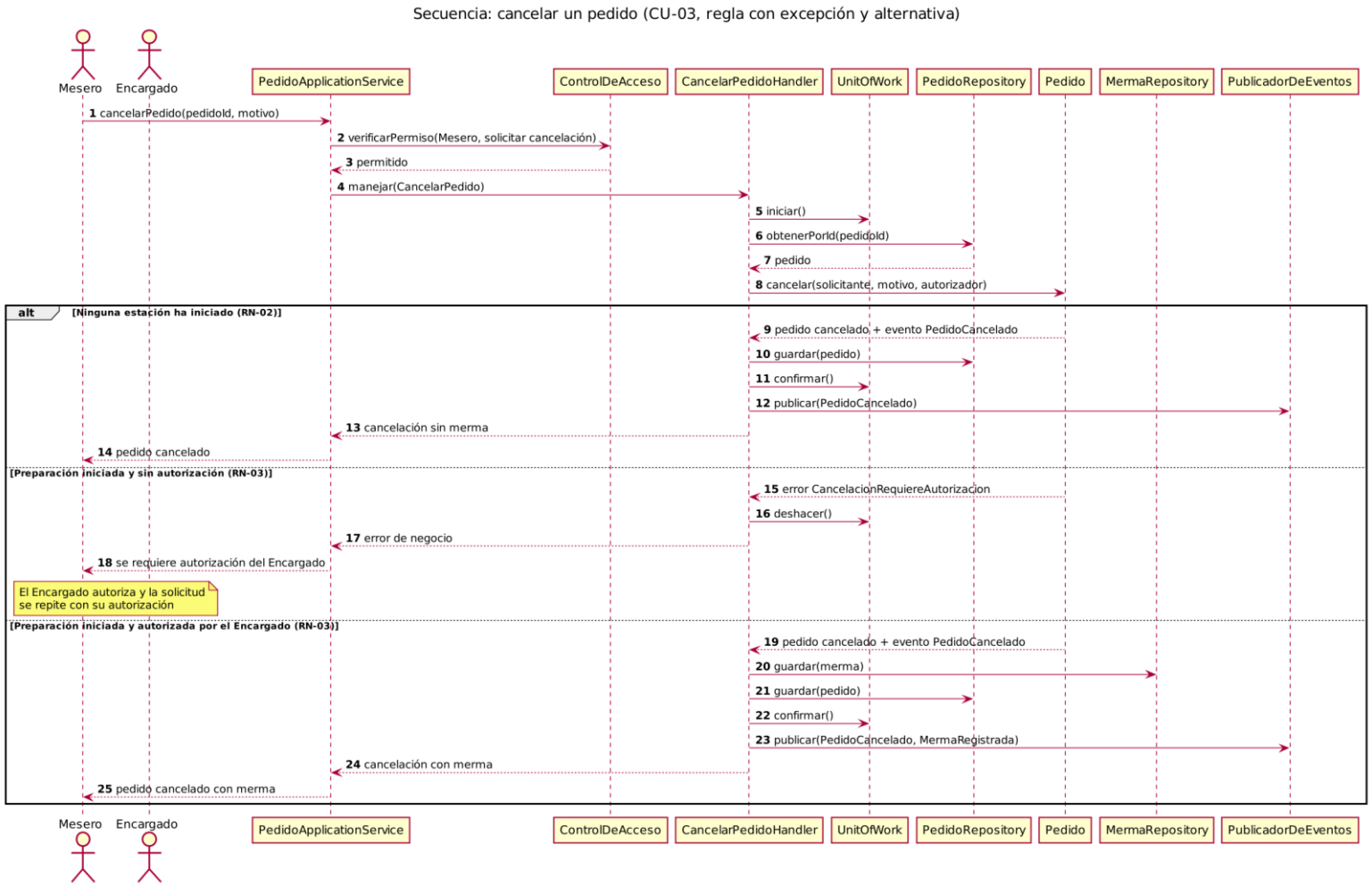


Reporte

# Código del diagrama de secuencia del registro en plantuml



**Regla con excepción y alternativa (CU-03):** cancelar un pedido. Tiene tres caminos: sin preparación iniciada se cancela directo (RN-02); con preparación iniciada y sin autorización se rechaza, se deshace la unidad de trabajo y no se publica ningún evento; con la autorización del Encargado se cancela, se guarda la merma junto con el pedido y se publican los eventos (RN-03).



Reporte

# Código del diagrama de secuencia de cancelación en plantuml

Home
What's New ?
Getting Started
Online Server
Running
F.A.Q.
Download
Forum
Theme
Preprocessing
Standard Library
PDF Guide
Hitchhiker's Guide

Secuencia Usecase Clase Actividad Component Estado Objeto Despliegue Timing Regex Network Wireframe Archimate Gantt MindMap WBS EBN

**New!** Render PlantUML diagrams directly inside GitHub with our official browser extension — No server. No tokens. No tracking. Zero permissions but clipboard. — Try it out and let us know what you think!

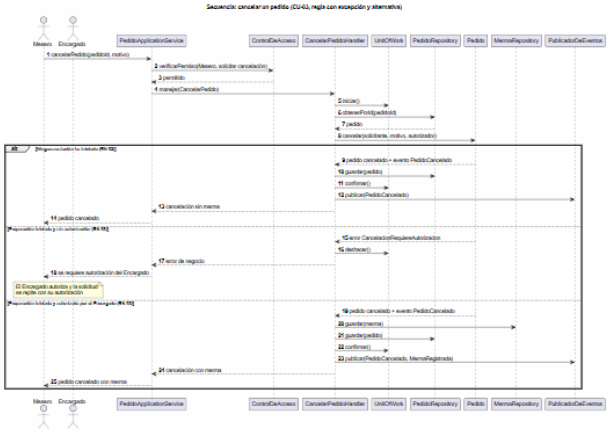
Add to Chrome Add to Firefox

*This editor runs entirely in your browser thanks to our JavaScript rendering engine. Edit the code on the left and see the diagram update instantly!*

```
participant "MermaRepository" as MR
participant "PublicadorDeEventos" as EV

Mesero -> PS : cancelarPedido(pedidoId, motivo)
PS -> CA : verificarPermiso(Mesero, solicitar cancelación)
CA --> PS : permitido
PS -> H : manejar(CancelarPedido)
H -> UoW : iniciar()
H -> PR : obtenerPorId(pedidoId)
PR --> H : pedido
H -> P : cancelar(solicitante, motivo, autorizador)

alt Ninguna estación ha iniciado (RN-02)
  P --> H : pedido cancelado + evento PedidoCancelado
  H -> PR : guardar(pedido)
  H -> UoW : confirmar()
  H -> EV : publicar(PedidoCancelado)
  H --> PS : cancelación sin merma
  PS --> Mesero : pedido cancelado
else Preparación iniciada y sin autorización (RN-03)
  P --> H : error CancelacionRequiereAutorizacion
  H -> UoW : deshacer()
  H --> PS : error de negocio
```



PlantUML de un vistazo

## Architecture Decision Records

Un ADR registra una decisión de arquitectura con su contexto, las alternativas que se consideraron, lo que se decidió y las consecuencias. Escribimos tres, cada uno con su estado, contexto, alternativas, decisión, consecuencias y trazabilidad hacia necesidades, reglas, historias y casos de uso.

Resumen de los ADR:

- **ADR-01, patrón de lógica de negocio:** Domain Model como principal y Transaction Script solo para marcar agotado. Consecuencia aceptada: más estructura inicial y un Pedido con mucho comportamiento.
- **ADR-02, Repository y Unit of Work:** interfaces en el dominio, implementaciones en infraestructura y una unidad de trabajo por solicitud. Consecuencia aceptada: sin control de concurrencia, por ser un sistema ficticio de una sola base.
- **ADR-03, CQRS básico:** nueve commands y cuatro queries con handlers y DTO, sobre una sola base de datos. Consecuencia aceptada: más clases, un handler por operación.

### ADR-01, patrón de lógica de negocio

| Campo | Contenido |
|---|---|
| **Título** | Domain Model como patrón de lógica de negocio, con Transaction Script solo para marcar productos agotados |
| **Contexto** | Sazón Urbano maneja 15 reglas de negocio, 6 roles y 3 canales de pedido. Varias reglas dependen del estado del pedido por estación (RN-02, RN-03, RN-12) y deben aplicarse igual sin importar el canal. Las promociones cambian con frecuencia (por ejemplo, el 2x1 de martes que hoy se aplica "de memoria") |
| **Alternativas** | Transaction Script: rápido de construir, pero duplicaría las reglas en cada flujo y cada cambio tocaría varios scripts. Table Module: muy bueno para reportes, pero no modela el ciclo de vida del pedido. Domain Model: reglas y estado juntos, a costa de más estructura inicial |
| **Decisión** | Domain Model como patrón principal. Transaction Script solo en CU-04a (marcar agotado), por ser una operación lineal. Table Module queda descartado en los 5 casos de uso |
| **Consecuencias** | Beneficios: una regla vive en un solo lugar y los cambios quedan localizados. Costos: más clases y estructura que con scripts. Riesgos: `Pedido` concentra mucho comportamiento (observación de la revisión SOLID). Trabajo posterior: los reportes (US-13 y US-17) podrían usar Table Module |
| **Trazabilidad** | NEC-04, NEC-05, NEC-11, NEC-12 · RN-01 a RN-06, RN-12, RN-14 · US-03, 05, 06, 07, 08, 09, 18 · CU-01 a CU-05 |

Reporte

ADR-02, Repository y Unit of Work

| Campo | Contenido |
|---|---|
| **Título** | Repositorios con interfaz en el dominio y una Unit of Work por solicitud |
| **Contexto** | El dominio no debe depender de SQL ni de la base de datos. La cancelación con preparación iniciada debe guardar `Pedido` y `Merma` juntos (RN-03). El BRD advierte que la red puede tener intermitencias. No hay base de datos real en este ejercicio |
| **Alternativas** | Servicios con SQL directo: acopla el negocio a la tecnología. Repository sin Unit of Work: riesgo de guardar solo una parte de los cambios. Repository + Unit of Work por solicitud**:** más piezas, pero con atomicidad |
| **Decisión** | Interfaces de Repository en el dominio (9 métodos en total) e implementaciones en infraestructura (en memoria para el código base). Una Unit of Work por solicitud**:** se confirma al final o se deshace todo, y los eventos se publican después de confirmar. No se implementa protección contra concurrencia, para mantener el diseño simple |
| **Consecuencias** | Beneficios**:** se puede cambiar de tecnología sin tocar el negocio, y `cancelarPedido` nunca deja un pedido cancelado sin su merma. Costos**:** más interfaces. Riesgos**:** sin control de concurrencia, dos usuarios podrían actuar sobre un estado desactualizado (por ejemplo, cancelar mientras cocina inicia una estación); se acepta por ser un sistema ficticio de una sola base. Trabajo posterior: si pasara a producción, agregar control de versiones sobre el pedido |
| **Trazabilidad** | NEC-01, NEC-04 · RN-03, RN-13 · Restricciones del BRD (red intermitente) · Hallazgos H4 y H5 de la revisión SOLID |

ADR-03 CQRS básico

| Campo | Contenido |
|---|---|
| **Título** | CQRS básico con Handlers, DTO de lectura y una sola base de datos |
| **Contexto** | Hay operaciones que cambian el estado (registrar, cancelar, asignar) y necesidades de solo lectura (resumen del pedido, menú disponible, promociones vigentes, repartidores disponibles). US-04 pide que lo agotado "deje de mostrarse" sin meter una regla extra al registrar pedidos. La guía no exige bases de datos separadas |
| **Alternativas** | Un solo modelo para leer y escribir: expondría el aggregate `Pedido` a pantallas y mezclaría responsabilidades. CQRS completo con bases separadas: complejidad que un restaurante único no justifica. CQRS básico: separación en el código, una sola base |
| **Decisión** | 9 Commands y 4 Queries, cada uno con su Handler. Las consultas dependen de interfaces de lectura y devuelven DTO, nunca el aggregate. Una sola base de datos |
| **Consecuencias** | Beneficios**:** responsabilidades claras, lecturas con el modelo adecuado y cumplimiento del principio I de SOLID. Costos**:** más clases (un handler por operación). Riesgos**:** duplicar conceptos entre el DTO y el aggregate. Trabajo posterior**:** los reportes pesados podrían separarse más adelante |
| **Trazabilidad** | NEC-03, NEC-08, NEC-09 · RN-05 · US-04, US-13, US-17 · Hallazgo H6 |

## Validación de la propuesta

Antes de entregar recorrimos cada caso de uso sobre el diseño completo para comprobar que fuera coherente: qué componente recibe la solicitud, dónde se valida cada regla, qué objetos cambian, cómo se persisten los cambios y qué resultado se devuelve. Después aplicamos las seis validaciones de la guía (trazabilidad, lenguaje, responsabilidad, consistencia, persistencia y transacción) y una más sobre el uso de la IA.

- Trazabilidad, responsabilidad, consistencia, persistencia y transacción: cumplidas.
- Lenguaje: no, porque el único desfase es Mesero y Caja respecto del BRD; está registrado y se corrige en la matriz de trazabilidad.
- Durante la validación corregimos cinco inconsistencias entre los diagramas y las etapas: un método de creación del Pedido que no existía en la clase, dos componentes sin consumidor, dependencias incompletas en dos servicios y dos métodos de cierre de entrega que faltaban en el Pedido.
- Quedan declarados como supuestos pendientes de validar: Mesero y Caja separados, que RN-06 prevalece sobre RN-12 al quitar una línea agotada, que el precio se conserva al capturar el pedido, el mecanismo de autorización del Encargado, los estados del pedido derivados del BPMN y que cada línea lleva la estación que la prepara.

| CU | ¿Quién recibe la solicitud ? | ¿Dónde se valida cada regla? | ¿Qué objetos cambian? | ¿Cómo se persiste? | ¿Qué se devuelve? |
|---|---|---|---|---|---|
| 01 Registrar pedido | PedidoApplicationService → RegistrarPedidoHandler | **Pedido:** RN-01, RN-14 y datos de domicilio. Permiso de rol: ControlDeAcceso | Pedido nuevo con sus líneas | PedidoRepository.guardar dentro de la Unit of Work; el evento PedidoConfirmado se publica **después** de confirmar | Id, total y estado. Error: MontoMinimoDomicilioNoAlcanzado |
| 02 Aplicar promoción | PedidoApplicationService → AplicarPromocionHandler | **Promoción** (vigencia, productos, descuento, vía `AplicadorDePromocion`) y **Pedido** (RN-04, una sola promoción) | `Pedido` (promoción aplicada y total) | PedidoRepository.guardar | Pedido con descuento. Errores: fuera de vigencia, no aplicable, ya tiene promoción |
| 03 Cancelar pedido | PedidoApplicationService → CancelarPedidoHandler | **Pedido:** RN-02 y RN-03 (¿alguna estación inició? ¿hay autorizador? ) | `Pedido` (cancelación) y `Merma` nueva si hubo preparación | `PedidoRepository` y `MermaRepository` **en una sola unidad de trabajo** | Cancelación con o sin merma. Error: CancelacionRequiereAutorizacion |
| 04 Producto agotado | DisponibilidadApplicationService.gestionarProductoAgotado | *Marcar:* ninguna regla (Transaction Script). *Quitar línea:* **Pedido** (RN-06, RN-12) | *Marcar:* registro de disponibilidad. *Quitar:* `Pedido` (línea, total, cancelación parcial) | `DisponibilidadRepository.marcarAgotado` o `PedidoRepository.guardar` | Producto no disponible con aviso a Mesero y Caja, o pedido actualizado |
| 05 Entrega | `EntregaApplicationService.gestionarEntrega` | **Entrega** y `AsignadorDeRepartidor` (disponibilidad, solo domicilio, RN-11) y **Pedido** (estado listo y cierre) | `Entrega`, `Repartidor` y, al cerrar, `Pedido` | Repositorios de Entrega, Repartidor y Pedido **en una sola unidad de trabajo** | Entrega asignada, en ruta, completada o fallida, según la solicitud |

Tabla de validación

| Validación | Pregunta | Resultado del equipo |
|---|---|---|
| **Trazabilidad** | ¿Cada componente importante responde a una necesidad, regla o caso de uso? | **Sí.** Los únicos componentes sin necesidad directa son de soporte (`PublicadorDeEventos`, `Dinero`) y se justifican por los eventos y los importes del dominio |
| **Lenguaje** | ¿Los mismos términos significan lo mismo en BRD, UML, código y ADR? | **No.** El único desfase es **Mesero y Caja**: el BRD, RN-05, US-08 y el glosario los tratan juntos o con otro alcance. Está registrado como H-01 y H-07, y se corrige en la matriz de trazabilidad |
| **Responsabilidad** | ¿Las reglas permanecen en el dominio o en el patrón elegido con una justificación? | **Sí.** Las reglas de negocio viven en `Pedido`, `Promoción`, `Entrega` y `Merma`. Las excepciones están justificadas: marcar agotado (Transaction Script) y permisos por rol (capa de aplicación) |
| **Consistencia** | ¿Los diagramas de clase y secuencia representan la misma arquitectura? | **Sí.** Los 15 participantes de las secuencias existen en el diagrama de clases, sin clases sueltas, y se corrigieron 5 inconsistencias |
| **Persistencia** | ¿El dominio evita depender directamente de SQL, ORM o infraestructura? | **Sí.** El dominio y la aplicación solo conocen interfaces; el SQL queda en las implementaciones |
| **Transacción** | ¿El límite de la operación y su manejo de error están definidos? | **Sí.** Una unidad de trabajo por solicitud: se confirma al final o se deshace todo. Los errores se dividen en de negocio y técnicos |

## Bitacora de IA

Usamos Claude (Anthropic) como apoyo siguiendo el ciclo que propone la guía: preparar el contexto, solicitar una propuesta con supuestos explícitos, compararla con las clases y el BRD, corregirla, validarla y registrarla. La IA propuso; el equipo decidió.

Errores de IA que detectamos y corregimos:

- Trató “Mesero / Caja” como un solo rol; el equipo los separó.
- Las primeras etapas no seguían el formato de la guía oficial; se rehicieron.
- Propuso nueve métodos de servicio en lugar de uno por caso de uso; se redujeron a cinco.
- Agregó protección contra concurrencia innecesaria; se eliminó.
- Usó Mermaid para los diagramas cuando la actividad indica PlantUML; se convirtieron.
- Los diagramas tenían inconsistencias con las etapas; se detectaron al verificar y se corrigieron.

Algunas desicioones que nosotros tomamos fueron:

Roles de Mesero y Caja separados, aprobación de los cinco casos de uso, el caso de producto agotado partido en dos con patrones distintos, mantener a Repartidor como aggregate propio, Service Layer con cinco métodos, quitar la concurrencia, usar PlantUML y aprobar las tres correcciones de SOLID.

Tabla de iteraciones de la bitácora

Reporte

| Objetivo | Prompt (resumen) | Respuesta útil | Error o supuesto | Decisión del equipo |
|---|---|---|---|---|
| Reunir lo esencial de las clases, la actividad y la U2 | Recopilar lo más importante de las 4 clases y de los archivos; después lo que haremos. No hacer todo en un documento: la IA propone y yo decido | Resumen de patrones, Service Layer, DDD, CQRS y SOLID; aclaraciones de la clase de la actividad; propuesta de 5 casos de uso; 4 posibles inconsistencias de la U2 | Mantuvo “Mesero / Caja” como un solo rol (así estaba en el BRD). Solo había visto la foto de instrucciones, no la guía de Teams | Se aprobaron los 5 casos. Mesero y Caja serán roles separados. Se aclararon H-02, H-03 y H-04; el BPMN se completa después |
| Elegir el patrón de lógica de negocio | Justificar bien por qué cada patrón y dónde se usa, de forma realista. | Tabla comparativa y decisión por caso: Domain Model como núcleo y Transaction Script solo para marcar agotado; Table Module descartado | Todavía sin la guía oficial: faltaban ID de caso, objetivo, información de entrada y resultado esperado | Se aprobó dejar el caso 4 partido en 4a y 4b |
| Mapa de dominio | Hacerlo tomando en cuenta todo lo indicado en clases, archivos e instrucciones | 5 aggregates, value objects, 2 Domain Services, eventos, glosario actualizado, decisiones dudosas y diagrama | Supuestos: el precio se conserva al capturar (H-09), RN-06 prevalece sobre RN-12 (H-08), estados del pedido derivados del BPMN (H-12). Riesgo de sobreingeniería en Repartidor y Merma | Repartidor se queda como aggregate propio**:** no se cambia si implica rehacer diagrama y tablas |
| Service Layer (versión 1) | Continuar con la etapa 4 | 3 servicios con 9 métodos | **Error:** no respetaba “un método por caso de uso” de la foto del profe. Lo detectaste al preguntar por qué eran 9 | Se eligió la opción B: 5 métodos, uno por caso de uso, para bajar la complejidad |
| Service Layer (versión 2) | Ajustarlo a 5 métodos sin complicar el sistema | 5 métodos en 3 servicios; los dos multipaso enrutan solicitudes | **Riesgo:** gestionarProductoAgotado y gestionarEntrega tienen nombres genéricos | **Se aceptó. Se verificó que el cambio** no afectó las etapas 1 a 3 |
| Repository y Unit of Work | Pasar a la etapa 5 | 6 repositorios (10 métodos), tabla de qué se guarda junto y manejo de errores | **Error:** agregué protección contra concurrencia sin que fuera necesaria. **Supuesto:** el repartidor queda libre al cerrar la entrega | Se quitó la protección contra concurrencia para no complicar el diseño |
| Commands y Queries | Pasar a la etapa 6; preguntó qué significa DTO | 9 Commands, 4 Queries, handlers, DTO y qué parte no necesita la separación | Supuesto**:** el menú disponible se lee desde un catálogo que no es aggregate | Se aprobó. Se aclaró el concepto de DTO |
| Revisión con SOLID | Pasar a la etapa 7 | 7 hallazgos; 3 correcciones: Promoción calcula su descuento (O), contrato común de repositorios (L), | Aclaré que Pedido concentra mucho comportamiento y que no conviene dividirlo | Se aprobaron las 3 correcciones y se pidió el texto corregido listo para copiar y pegar |

| | | | | |
|---|---|---|---|---|
| | | separar lectura de disponibilidad (I) | | |
| Diagramas en PlantUML y verificación | Convertir, indicar dónde va el código y verificar antes de seguir | 3 .puml y sus .png, renderizados y revisados | **Errores propios detectados al verificar:** Pedido.crear no existía en la clase; PromocionRepository y AplicadorDePromocion no tenían consumidor; dependencias incompletas en dos servicios | Se aceptaron las correcciones y se descartaron los archivos de Mermaid |
| ADR | Pasar a ADR | 3 ADR con título, estado, contexto, alternativas, decisión, consecuencias y trazabilidad | **Supuesto:** el ADR-04 de Mesero y Caja se dejó como opcional | Se aprobaron los 3 ADR y se siguió a la etapa 9 |
| Validar la propuesta | Pasar a la etapa 9 | Recorrido de cada caso de uso, tabla de validación, pendientes y lista final | **Error propio:** Pedido seguía sin marcarEntregado() y marcarEntregaNoCompletada(); se corrigió. También omití la fila "IA" de la tabla de la guía, y la noté después | Se aprobó y se pasó a los entregables de cierre |
| Matriz de trazabilidad | Avanzar a la siguiente etapa | 5 tablas: matriz principal, cobertura de la U2, permisos por rol, ajustes de Mesero y Caja, y hallazgos | **Supuestos:** el Mesero solo captura en mesa y la Caja los demás canales; los permisos de las consultas son propuesta de la IA | Se aprobó y se pasó a la bitácora |

## Errores cometidos por la IA

| # | Error | Cómo se detectó | Corrección |
|---|---|---|---|
| 1 | Tomó "Mesero / Caja" como un solo rol | Lo advertiste tú, por conocimiento del negocio | Roles separados y ajustes a la U2 (matriz, tabla 4) |
| 2 | Primeras etapas sin el formato de la guía | Al subir la guía de Teams y compararla | Se rehicieron las etapas 1 y 2 en el formato oficial |
| 3 | 9 métodos de servicio en lugar de uno por caso de uso | Cuestionaste el diseño | Reducido a 5 |
| 4 | Protección contra concurrencia innecesaria | Pediste menos complejidad | Se eliminó |

| | | | |
|---|---|---|---|
| 5 | Diagramas en Mermaid cuando la actividad indica PlantUML | Lo notaste al querer descargarlos | Conversión a PlantUML |
| 6 | Diagramas inconsistentes con las etapas (`crear`, clases sin uso, dependencias incompletas) | Verificación cruzada de clases y secuencias | Corregidos y re-exportados |
| 7 | `Pedido` sin los métodos de cierre de entrega | Recorrido de los casos de uso en la etapa 9 | Agregados al diagrama |
| 8 | Omisión de la fila "IA" en la tabla de validación | Revisión de la guía al armar la matriz | Se completa con esta bitácora |

## Tabla de decisiones del equipo

| **Decisión** | **Efecto** |
|---|---|
| Mesero y Caja como roles separados | Cambió permisos, glosario y user stories |
| Aprobar los 5 casos de uso | Definió el alcance del diseño |
| Partir CU-04 en 4a (Transaction Script) y 4b (Domain Model) | Patrón híbrido justificado |
| Mantener Repartidor como aggregate propio | Modelo sin cambios |
| Service Layer con 5 métodos | Fiel a la foto y menos complejo |
| Quitar la concurrencia | Diseño más simple |
| Usar PlantUML | Cumple lo que indica la actividad |
| Aprobar las 3 correcciones de SOLID | Mejoró extensibilidad, sustitución y segregación |

## Conclusiones

### Emmanuel Carrillo Loera

De lo que más me ayudo a llevar a cabo la actividad fue entender que la clave está en decidir dónde vive cada regla: por ejemplo, que lo de cancelar un pedido dependa de si una estación ya empezó a cocinar y que eso lo decida el pedido y no el servicio que lo llama.

Trabajar con la IA me ayudo a acelerar el proceso, pero se equivocó varias veces. Es cierto que con la IA solo se propone, pero uno tiene que revisar al final.

También me gustó que, al decidir cosas como separar al mesero de la caja, tuvimos que anotar que era un supuesto y no darlo por hecho. Lo confuso fue no complicar de más el diseño, y por eso quitamos cosas como el control de concurrencia.

### Fatima Alejandra Cavazos Serrato

En esta actividad pasamos del análisis del negocio, que en nuestro caso es “sazón urbano” a una propuesta antes de programar. Con los casos de uso nos apoyamos para definir cómo funcionaría y en qué momento se aplican las reglas del negocio, utilizamos diferentes patrones, los cuáles nos ayudan a que cada cambio se guarde y haya una separación de cada diseño, de igual manera se realizaron ajustes para que el diseño mejore.

### Edgar Daniel del Ángel Hernández

El desarrollo de este proyecto para Sazón Urbano demostró que es muy útil organizar y diseñar el sistema antes de empezar a programar. Al elegir el modelo de dominio adecuado, logramos que las reglas importantes del restaurante —como los pedidos mínimos a domicilio, las cancelaciones con autorizaciones o el control de mermas— queden guardadas directamente en las reglas del sistema y funcionen igual sin importar si el pedido es por mesa, teléfono o domicilio.

Además, separar las tareas de guardar datos, procesar operaciones y consultar información nos permitió mantener un código ordenado, flexible y fácil de mantener a futuro.